

AIM2- Preparations

DAY 4 — Functions & Modular Python

You'll learn:

- What functions are
- Why functions are important
- `def`
- Parameters
- Arguments
- `return`
- Local vs global variables
- Default parameters
- Multiple parameters
- Calling functions
- Reusing functions
- Organizing a program into functions

And then we're going to take your **Student Gradebook** and turn this:

```
one giant program

into:

main()
|
├─ get_student_data()
├─ calculate_total()
├─ calculate_average()
├─ calculate_grade()
└─ display_student()
```

What we're learning today

By the end of Day 4, you should understand:

1. What a function is
2. Why we use functions
3. `def`
4. Calling a function
5. Parameters
6. Arguments
7. `return`
8. Multiple parameters
9. Default parameters
10. Local vs global variables
11. Functions calling other functions
12. How to break a project into smaller functions

And we'll finish with a **Student Gradebook refactoring project**.

1 What is a function?

Imagine you have this code:

```
total = math + python + ai
average = total / 3
```

You might need this calculation for:

- Krishna
- Rahul
- Priya
- 100 other students

Would you copy and paste it 100 times?

✗ Absolutely not.

Instead, create a function.

```
def calculate_average(math, python, ai):  
    total = math + python + ai  
    average = total / 3  
    return average
```

Now you can reuse it:

```
krishna_average = calculate_average(95, 88, 65)  
rahul_average = calculate_average(82, 91, 78)
```

That's the power of functions.

2 Basic function syntax

The basic structure is:

```
def function_name():  
    # code
```

Example:

```
def say_hello():  
    print("Hello Krishna!")
```

But notice something important:

The function doesn't execute just because you created it.

You have to call it:

```
say_hello()
```

Output:

Hello Krishna!

Think:

```
def say_hello()
```

↓

Create the function

```
say_hello()
```

↓

Run the function

3 Why def?

def means:

Define a function.

Example:

```
def greet():  
    print("Hello!")
```

You're telling Python:

Create a function called greet.

4 Function with a parameter

Now let's make it useful.

```
def greet(name):  
    print(f"Hello, {name}!")
```

Call it:

```
greet("Krishna")
```

Output:

Hello, Krishna!

And:

```
greet("Rahul")
```

Output:

Hello, Rahul!

The function is reusable.

5 Parameter vs Argument

This terminology is important.

Look at:

```
def greet(name):  
    print(f"Hello, {name}!")
```

name is called a **parameter**.

When you do:

```
greet("Krishna")
```

"Krishna" is an **argument**.

Think:

```
def greet(name)  
    ↑
```

parameter

```
greet("Krishna")
```

↑

argument

Don't worry if these words feel similar at first. You'll get used to them.

6 Multiple parameters

You can have more than one.

```
def introduce(name, age):  
    print(f"My name is {name} and I am {age} years old.")
```

Call:

```
introduce("Krishna", 28)
```

Output:

My name is Krishna and I am 28 years old.

Another:

```
introduce("Rahul", 30)
```

7 return — VERY IMPORTANT ★

This is probably the most important concept of today's lesson.

Consider:

```
def add(a, b):  
    print(a + b)
```

If you do:

```
result = add(10, 20)
```

What is result?

It's:

None

Because the function printed the answer but didn't **return** it.

Instead:

```
def add(a, b):  
    return a + b
```

Now:

```
result = add(10, 20)
```

```
print(result)
```

Output:

30

Think about it this way:

```
print():
```

Show something to the screen.

return:

Give a value back to the code that called the function.

This distinction is extremely important.

8 print() vs return

print()

```
def add(a, b):  
    print(a + b)
```

The function displays:

30

But it doesn't give the result back.

return

```
def add(a, b):  
    return a + b
```

Now:

```
result = add(10, 20)
```

result contains:

30

So:

<pre>print() → display return → send value back</pre>

🔥 Remember this.

Our Gradebook example

Let's create:

```
def calculate_total(math, python, ai):  
    return math + python + ai
```

Then:

```
total = calculate_total(95, 88, 65)  
  
print(total)
```

Output:

248

Now:

```
def calculate_average(math, python, ai):  
    total = math + python + ai  
    return total / 3
```

Then:

```
average = calculate_average(95, 88, 65)  
  
print(average)
```

Output:

82.66666666666667

You can format it:

```
print(f"{average:.2f}")
```

Output:

82.67

10 Function for grade

Let's make another function.

```
def calculate_grade(average):  
  
    if average >= 90:  
        return "A"  
    elif average >= 80:  
        return "B"  
    elif average >= 70:  
        return "C"  
    elif average >= 60:  
        return "D"  
    else:  
        return "F"
```

Now:

```
grade = calculate_grade(82.67)  
  
print(grade)
```

Output:

B

Notice something beautiful here.

We separated responsibilities:

```
calculate_total()  
    ↓  
calculate_average()  
    ↓  
calculate_grade()
```

Each function does **one job**.

This is a very important software engineering principle.

1 1 Function for pass/fail

We can create:

```
def check_result(average):  
  
    if average >= 60:  
        return "PASSED"  
    else:  
        return "FAILED"
```

Then:

```
result = check_result(82.67)  
  
print(result)
```

Output:

PASSED

Look at what we've built

We now have:

```
calculate_total()  
    ↓  
calculate_average()  
    ↓
```

```
calculate_grade()
```



```
check_result()
```

Instead of one giant block of code.

This is called **modular programming**.

1 2 Default parameters

You can give a parameter a default value.

```
def greet(name="Student"):  
    print(f"Hello, {name}!")
```

Now:

```
greet("Krishna")
```

Output:

```
Hello, Krishna!
```

But:

```
greet()
```

Output:

```
Hello, Student!
```

Because "Student" is the default.

1 3 Multiple return values

Python can return more than one value.

Example:

```
def calculate_scores(math, python, ai):  
  
    total = math + python + ai  
    average = total / 3  
  
    return total, average
```

Then:

```
total, average = calculate_scores(95, 88, 65)  
  
print(total)  
print(average)
```

Output:

```
248  
82.66666666666667
```

This is very useful.

1 4 Local variables

Consider:

```
def calculate_total():  
    math = 95  
    python = 88  
    ai = 65
```

```
return math + python + ai
```

The variables:

```
math  
python  
ai
```

exist inside the function.

They're local variables.

You can't normally do:

```
print(math)
```

outside that function.

1 5 Global variables

Example:

```
program = "AIM2"  
  
def show_program():  
    print(program)  
  
show_program()
```

program was created outside the function, so it is a global variable.

1 6 Functions can call other functions

This is where things get interesting.

```
def calculate_total(math, python, ai):  
    return math + python + ai  
  
def calculate_average(math, python, ai):  
    total = calculate_total(math, python, ai)  
    return total / 3
```

Notice:

```
calculate_average()  
    ↓  
calculate_total()
```

One function calls another.

That's normal in real applications.

★ 1 7 Now let's refactor your Student Gradebook

Think about our architecture:

```
Student  Gradebook  
    |  
    ├── calculate_total()  
    |  
    ├── calculate_average()  
    |  
    ├── calculate_grade()  
    |  
    ├── check_result()  
    |  
    └── display_student()
```

That's much cleaner.

Exercise 1 — Simple Functions

Create:

```
greet()
```

Output:

```
Hello, welcome to AIM2 Python preparation!
```

Then create:

```
greet_student(name)
```

Example:

```
greet_student("Krishna")
```

Output:

```
Hello Krishna!
```

Exercise 2 — Calculator Functions

Create these functions:

```
add(a, b)
```

```
subtract(a, b)
```

```
multiply(a, b)
```

```
divide(a, b)
```

For example:

```
result = add(10, 5)
```

```
print(result)
```

Expected:

15

And:

```
result = multiply(10, 5)
```

Expected:

50

Exercise 3 — Student Functions

Create these four functions:

```
calculate_total(math, python, ai)
```

```
calculate_average(math, python, ai)
```

```
calculate_grade(average)
```

```
check_result(average)
```

Test them with:

```
Math = 95
```

```
Python = 88
```

```
AI = 65
```

Expected:

```
Total: 248
```

```
Average: 82.67
```

```
Grade: B
```

```
Result: PASSED
```

★ Exercise 4 — BIG DAY 4 PROJECT

Now take your **Day 3 Student Gradebook** and refactor it.

Use:

```
students = [  
    {  
        "name": "Krishna",  
        "math": 95,  
        "python": 88,  
        "ai": 65  
    },  
    {  
        "name": "Rahul",  
        "math": 82,  
        "python": 91,  
        "ai": 78  
    },  
    {  
        "name": "Priya",  
        "math": 90,  
        "python": 94,  
        "ai": 89  
    }  
]
```

Your program should use functions rather than putting all the logic directly inside the loop.

Something conceptually like:

```
for student in students:  
  
    total = calculate_total(...)  
  
    average = calculate_average(...)  
  
    grade = calculate_grade(...)
```

```
result = check_result(...)
```

```
display...
```

Expected result

```
=====
          STUDENT GRADEBOOK
=====
```

```
Name: Krishna
Total: 248
Average: 82.67
Grade: B
Result: PASSED
```

```
-----
```

```
Name: Rahul
Total: 251
Average: 83.67
Grade: B
Result: PASSED
```

```
-----
```

```
Name: Priya
Total: 273
Average: 91.00
Grade: A
Result: PASSED
```

Your Day 1 → Day 4 progress

Look at what you've accomplished:

DAY 1

Python basics

Variables

Data types

Strings

Input

Arithmetic

if/else

↓

DAY 2

Lists

Dictionaries

Loops

Student Gradebook

↓

DAY 3

Control Flow

if / elif / else

for loops

Conditions

Grade calculation

↓

DAY 4

Functions

Parameters

Arguments

Return

Modular programming

Gradebook refactoring